Structs and Classes, OOP

CS 1622

Jarrett Billingsley

Class Announcements

- yet another "real example" of adding a language feature
  - though you already did the lexing, parsing, and some of the semantic analysis for this stuff in the projects…
  - so we'll kinda skip that stuff!

Structured types

Type survey

- primitive types can only get you so far.
  - remember these are things like **`int`**, **`bool`**, and **`char`**.
- **arrays** give the programmer a lot of problem-solving power.
  - even in very simplistic languages (like old BASIC dialects), arrays give you the power of **indirection** and **dynamically-sized data structures,** which you can't do with primitives.
- but more complex problems will require the user to define their **own types** suited to solving the problem.
  - and if we want to let the user define their *own* types, the most obvious place to start is with… ~~classes~~ ~~structs~~ **tuples!**

Tuples??

- a **tuple** is an ordered sequence of **heterogeneous types.**
  - heterogeneous = "they can be different"
- typically, tuple types are written in parens, like `(T1, …, Tn)`
- you could use them to represent…
  - coordinates in an *n*-dimensional space: `(10, 4)`
    - this one's type is `(int, int)`
  - the absence of a value: `()`
    - this is also called **unit** or, in our language, **void**
  - simple **objects:** `("Billingsley", "Jarrett", 35)`
    - this one's type is `(string, string, int)`
  - maybe you can see where this is going…

Operations on tuples

- like arrays, we can **index** them to access their values:
  - `t[2] = 34;`
- *unlike* arrays, we know their length **statically…**
  - but each element can be a **different type!**
  - therefore: **we can only index them with *constant* values.**
- what if we had a way of defining named constants?

```
const LNAME = 0;
const FNAME = 1;
const AGE   = 2;
let me = ("Billingsley", "Jarrett", 35);
me[AGE] = 36; // next year..
```

- 

...yeah okay that's basically a struct isn't it

- a **struct** or **record type** is like a class that *only* has data.
- tuples and structs are almost the same thing, except...
  - tuples' fields are **anonymous** while structs' fields are **named.**
- it's probably pretty obvious now how structs work:

```
struct Point {
  x: int, y: int
}
```

≈

```
(int, int)
```

```
let p = new Point(3, 8);
```

≈

```
let p = (3, 8);
```

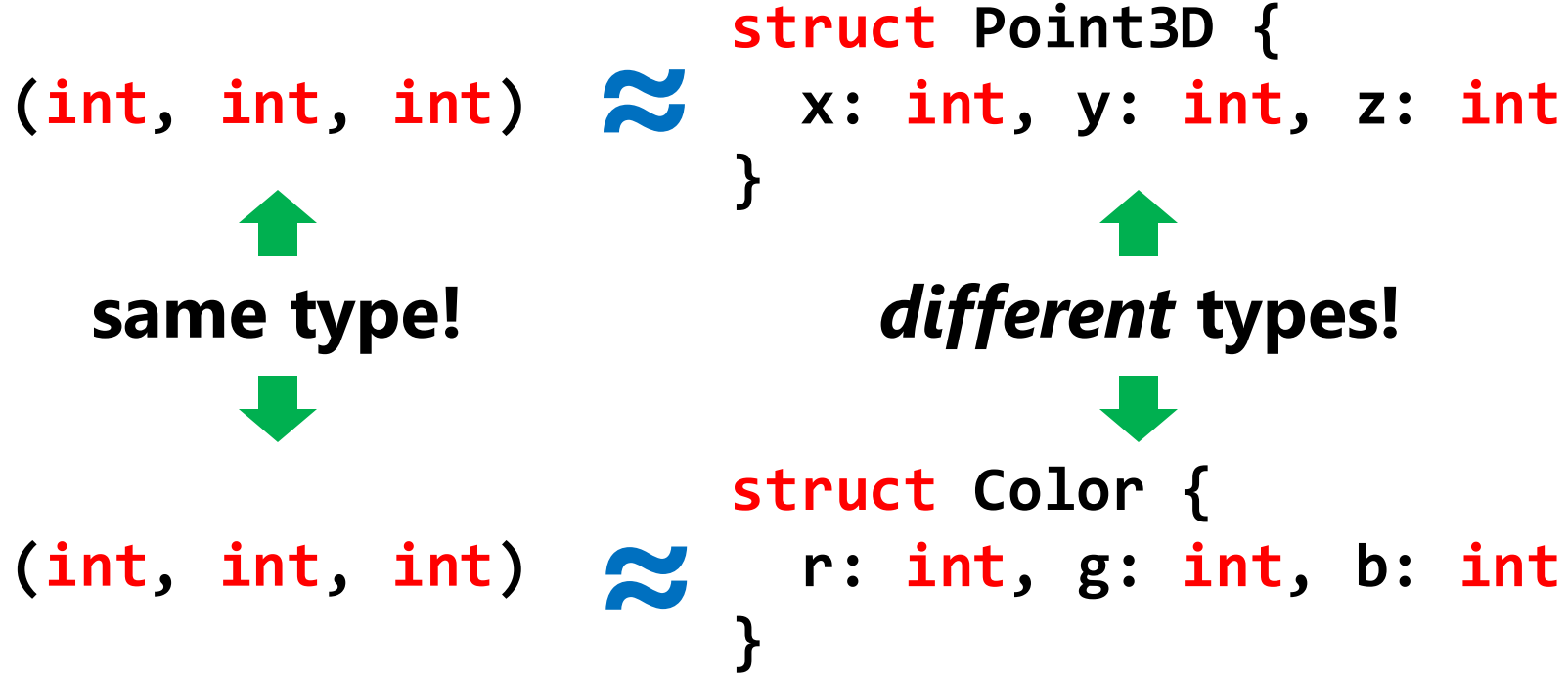
```
p.x = p.x + 1;
```

≈

```
p[0] = p[0] + 1;
```

One important difference

- in most languages that have both structs and tuples:
  - tuples have **structural identity,** but structs have **nominal identity**
- what *that* means is:



Value types and reference types

- *all* variables contain bit patterns.
- for **value type** variables, that bit pattern *is the value*.
  - examples include `int`, `float`, `char`, and `bool`.
- for **reference type** variables, that bit pattern is *a memory address*.
  - examples include Java arrays and objects.
- **assigning** from one variable to another **copies the bit patterns.**
- for **value types,** that means you *copy the entire value*.

```
int x = 5;
int y = x; // now we have two 5s!
```

- but for **reference types, you only copy *the address.***

```
Object x = new Object();
Object y = x; // still only one Object...
```

Decision time!

- Rust, C, C++, and C# consider structs to be *value types*. In Rust:

```
let s = Point { x: 3, y: 8 };
let t = s; // now we have two Points, both {3, 8}!
```

- in these languages, this is nonsense: `struct Node { next: Node }`
  - how can something contain itself??
- Java **doesn't let the programmer create their own value types.**
  - so, this is fine: `class Node { Node next; }`
  - because `Node next;` declares a *reference* variable.
- so: should Truss's structs be **value types,** or **reference types?**
- well, for "simplicity," let's stick with **reference types.**
  - reference types let us **solve more problems** than value types do, just sometimes with more space/time overhead.
- but if we have references, then we probably need... `null`.

The billion dollar mistake: null

- `null` is a *special value* you can put into a reference variable to indicate that it is uninitialized or "not pointing to anything."
- trying to perform any operation besides assignment on `null` will lead to a **runtime error** (a crash).
- unfortunately, *most of the time, you do not want references to be null.*
  - so by putting `null` in a language, and *not* requiring checks before using a reference, you are adding a **huge number** of potential crashes into virtually every program.
- also it's just *weird* when it comes to typechecking.
  - it's the only value that can be put into *multiple different types.*
  - `String s = null; Object o = null; int[] a = null;`
- so `null` does not have a type of its own! type theory calls it a **bottom type,** because it is at the bottom of the type hierarchy and acts as a subtype of all other types.

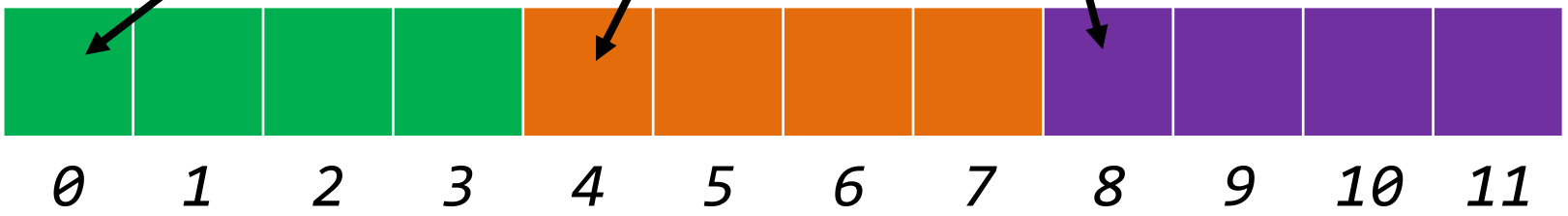
Struct Runtime Representation

Data layout

- how are structs (and classes?) going to be represented **in memory?**
  - arrays work by putting the values next to each other.
  - **okay, let's do that.**

```
struct Point3D { x: int, y: int, z: int }
```

assuming **int** is 32 bits (4 Bytes)...



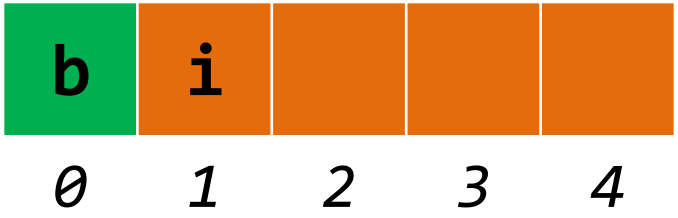
these numbers are the **offsets** from the beginning of the struct.
(assuming we're using a *byte-addressable* machine)

they say how far "into the struct" each field is.

Not so fast, Mister Gordon!

- since **bool** takes up 1 byte, what if we had a struct like this:

```
struct Hmm { b: bool, i: int }
```



this is a totally valid way of doing it.

unfortunately, **lots of CPU architectures *hate this*.**

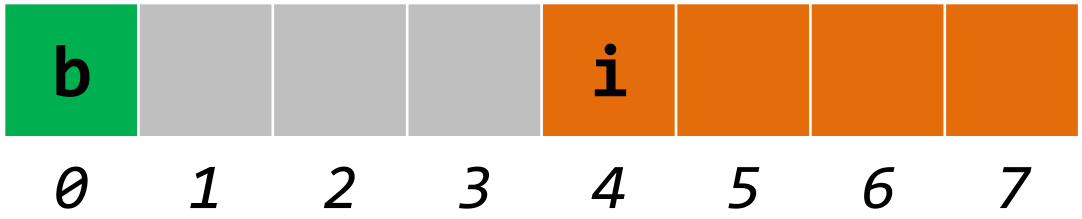
CPUs like values to be **aligned.** that means: the memory address in an *n*-byte load/store **must be a multiple of *n*.**

x86 CPUs will access unaligned values *very slowly*.
MIPS CPUs will crash. (unless you use the much slower unaligned loads/stores)

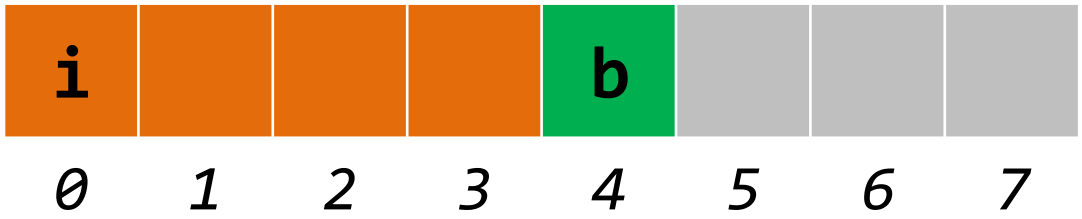
so, we need to insert **padding** (extra unused bytes) into our structs to ensure each field is properly aligned.

Struct padding

- we place each field **at an offset that is a multiple of its alignment.**
- then, the struct's size **as a whole** is rounded up to the **largest alignment of any field.** (this is important for arrays of structs.)
- so for this: `struct Hmm { b: bool, i: int }`



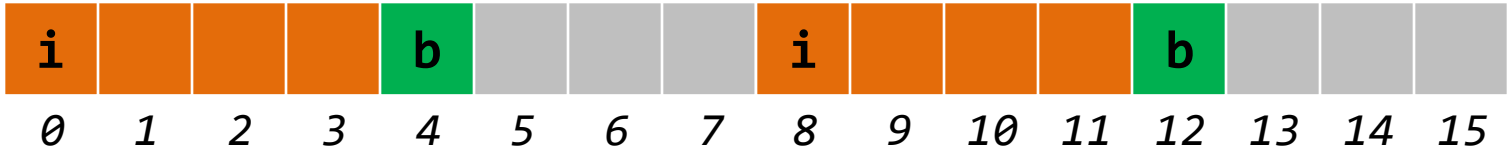
and if we **swapped b and i:**

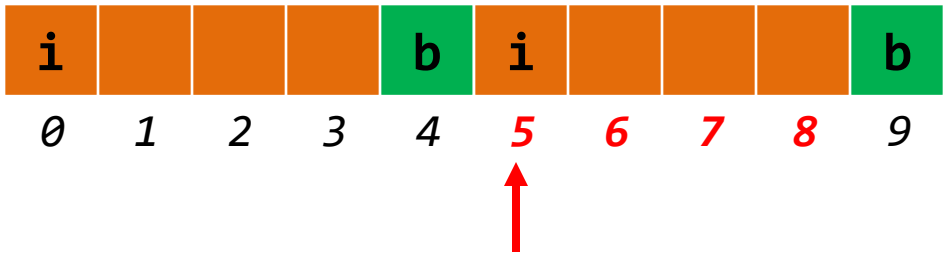


either way, the struct is **8 bytes,** and it must be **4-byte aligned** because of the **int.**

Wait, why does the *whole* struct need to be padded?

- strictly speaking, for our language where structs are *reference types*, the structs **don't** need padding at the end.
- but in languages where they're *value types*, it's crucial for maintaining alignment in **arrays of structs.** consider these arrays of two structs:





without the end-padding, this int field of the second item is now misaligned.

the **memory allocator** will have to allocate things at aligned memory addresses as well, for this to work.

Anyway... allocation

- now we know the **size** and **alignment** of each struct.
- we'll need a **runtime library function** to dynamically allocate structs.
  - let's call it **`rt$new(size: int)`**. it'll return an *address*.
  - to keep it simple, it'll always align things to 4 bytes.

```
struct Thing {     // size: 12
  name:   string,
  in_use: bool,
  flag:   bool,
  count:  int,
}

let t = new Thing();
```

allocations get generated as calls to this function.

```
addi sp, sp, -4
li   s0, 12
sw   s0, 0(sp) # size
jal  rt$new
sw   v0, -24(fp) # t
```

Accessing fields

- we use the fields' **offsets** and **types** to generate code for field access.

```
struct Thing {
  name:   string, // offs: 0
  in_use: bool,   // offs: 4
  flag:   bool,   // offs: 5
  count:  int,    // offs: 8
}

let t = new Thing();

t.in_use = true;
```

```
lw s0, -24(fp) # t
li s1, 1       # true
sb s1, 4(s0)   # t.in_use = true
```

```
t.count  = 20;
```

```
lw s0, -24(fp) # t
li s1, 20
sw s1, 8(s0)   # t.count = 20
```

to access a field, we add its **offset** to the struct instance's address, then use the right "flavor" of load/store.

OOP

What *is* OOP (object-oriented programming)?

- we might gesture vaguely in Java's direction and say "it's like that"
  - but "Java-flavored OOP" isn't the only kind…
- C has structs, but it **isn't** "OOP." C++ has "classes", and it **is** "OOP."
- the core idea of OOP is packaging **data** alongside **operations on that data** into a single, indivisible entity.
  - so: the main thing that distinguishes C++ from C… is **methods.**
- a **method call** is a special kind of function call:

  ```
  obj.method();
  ```

  - method calls can be more complex than "regular" function calls.
- other OOP concepts, which are not *required* but are common, include **inheritance, subtyping, interfaces, privacy, constructors,** and **run-time type identification.**

Methods

- **methods** are special functions which are associated with an object.
- they take the object they are called on **as an argument.**

```
struct Point {
  x: int,
  y: int
  fn to_string(): string {
    return to_string(this.x) + ", " + to_string(this.y);
  }
}

let p = new Point(3, 8);
println_s(p.to_string());
```

this is passed *implicitly,*
but it's still an argument.

here, the method call is **syntactic sugar** for something like:
`Point.to_string(p)`

this is **static dispatch,** because we can determine which method to call at compile-time.

Constructors

- a **constructor** is just a method that is called on an instance of an object immediately after allocating it, to let you initialize its fields.

```
class C {
  String s;

  C() {
    // s == null
    s = "hi";
  }
}

C c = new C();
```

in this Java code, `new C()` does the following:

1. allocates an instance of C
2. initializes all its fields to default values*
3. calls the constructor on that instance

but Java doesn't really *enforce* that you do anything useful in the constructor, and before it completes, the object may be in an **inconsistent state** (i.e. calling methods on `this` may behave erratically or crash).

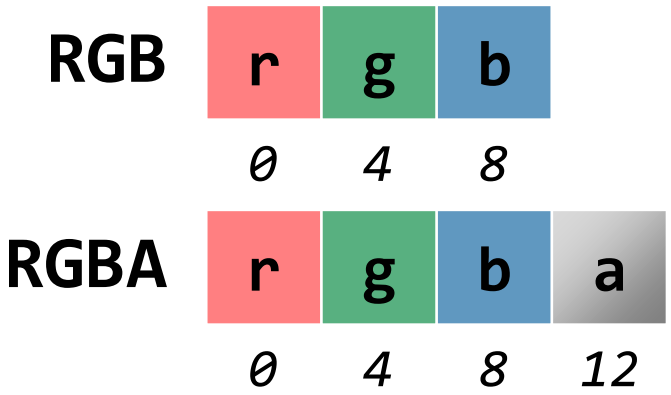
Data inheritance

- first, let's look at a simpler kind of inheritance, which *just* inherits data fields **without** inheriting methods, and does not imply subtyping.

```
struct RGB { r: int, g: int, b: int }
struct RGBA : RGB { a: int }
let color = new RGBA(255, 127, 0, 95);
```

the base class's fields are **copied** into the derived class's definition.

we represent this in memory by **prefixing:** the beginning of an RGBA *is an RGB*.



reusing *only* the data is limited in usefulness. reusing **methods** is much more useful.

Method inheritance

- method inheritance takes advantage of the **prefixed** representation.
  - base class methods can be called on derived class instances, because the base class fields are *at the same offsets*.

```
struct Animal {
  species: string
  fn to_string(): string {
    return "Animal(" + this.species + ")";
  }
}
struct Cat : Animal { whiskers: int }
let c = new Cat("felis cattus", 15);
println_s(c.to_string());
```

this is **still static dispatch:** the method call is sugar for **`Animal.to_string(c)`**.

we can now say that **Cat** is a **subtype** of **Animal.**

Where it starts getting crazy

- what if we want to **specialize `to_string()`** in the **`Cat`** struct?

```
struct Animal {
  species: string
  fn to_string(): string {
    return "Animal(" + this.species + ")";
  }
}
struct Cat : Animal {
  whiskers: int
  fn to_string(): string {
    return "meow meow meow!";
  }
}
let c = new Cat("felis cattus", 15);
println_s(c.to_string());
```

the compiler *might* be able to statically determine in *this* case that it should call **`Cat.toString(c)`**.

but **it can't determine that in the general case.**

The problem

- since a subtype can be used anywhere a base type is expected...
  - a base type variable can contain **different types of values.**
- and since subtypes can have **different implementations** of the **same methods** as the supertype...
  - you can't know what subtype implementation to call until runtime!

```
fn print(a: Animal) {
  // how does this call to
  // to_string know which
  // implementation to use?
  println_s(a.to_string());
}

print(new Cat());
print(new Dog());
print(new Horse());
```

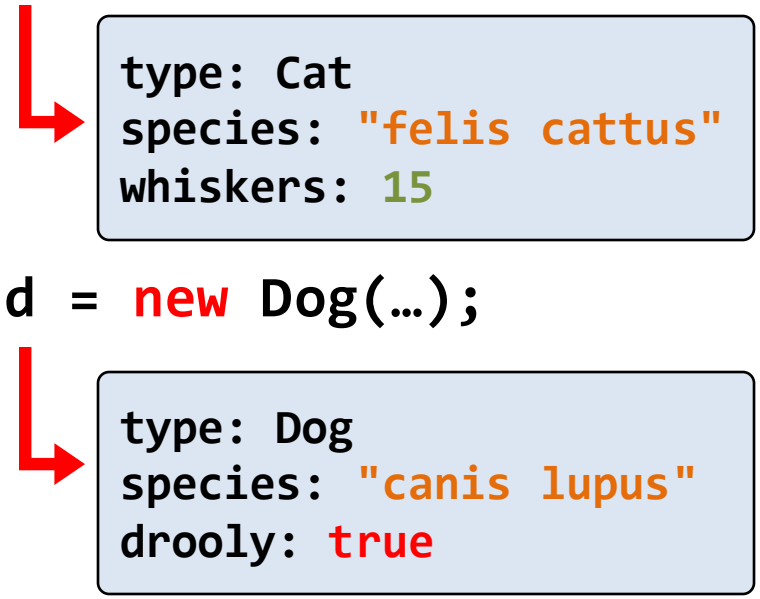
the problem (as it often is) is that **we lose type information** in the translation from the source language to the target language.

so, let's... *not* do that. **let's keep some of that type info.**

Virtual Methods and Dynamic Dispatch

RTTI (Run-time Type Identification)

- the solution is to make each class instance **carry an ID card.**
  - this is implemented as a "secret" field in **every** class instance.




```
fn print(a: Animal) {
  println_s(a.to_string());
}
```

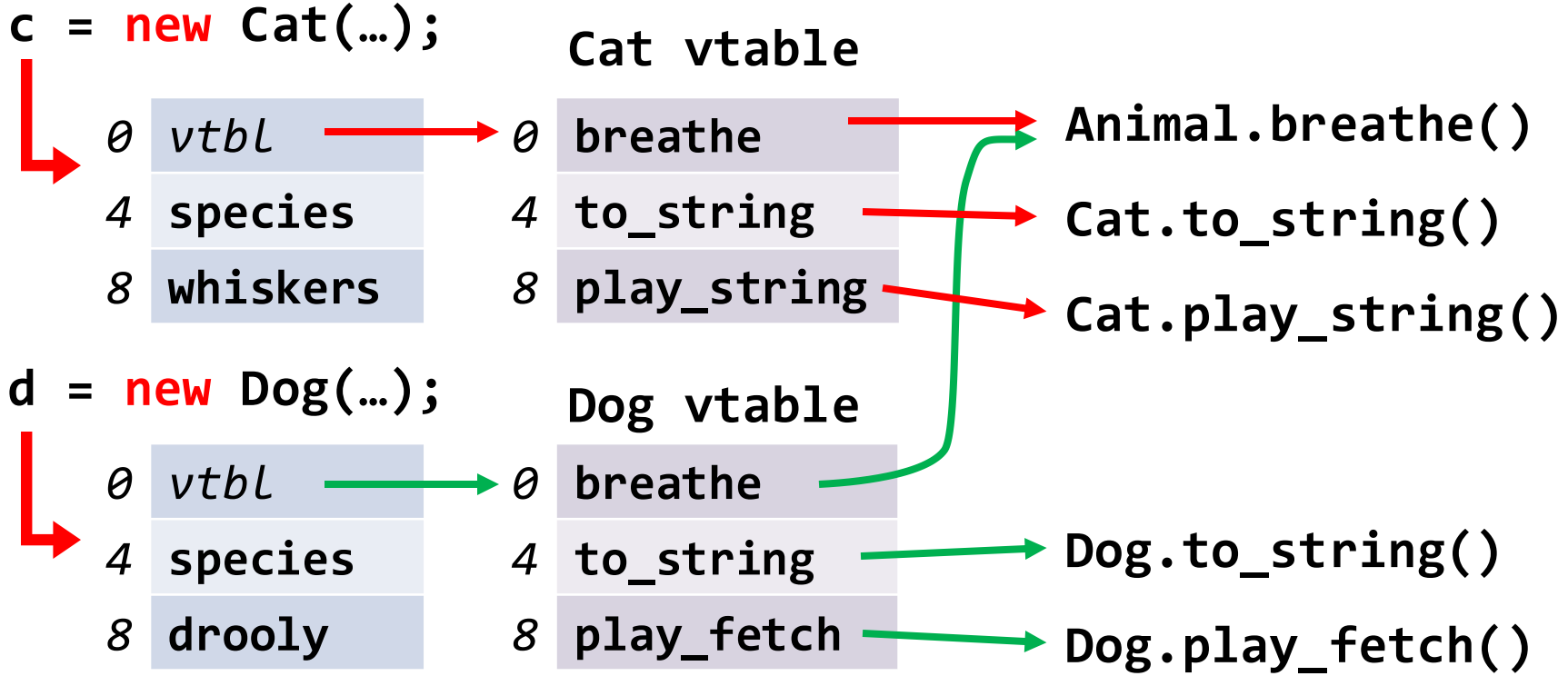
this method call *could* be implemented something like:

```
if a.type == Cat {
  Cat.to_string(a)
} else if a.type == Dog {
  Dog.to_string(a)
}
```

but that would be horribly inefficient.

Virtual method tables (vtables)

- to implement this **efficiently**, and to allow an **unlimited** number of subtypes, the "ID card" is a pointer to **an array of function pointers.**



Virtual method calls

- to call a virtual method, we **index the table** to get the address...
  - and do an **indirect function call** to that address.

```
fn print(a: Animal) {
  println_s(a.to_string());
}
```

this method call becomes:

```
a.vtbl[1](a)
```

or in assembly:

```
addi sp, sp, -4
lw   s0, -16(fp) # s0 = a
sw   s0, 0(sp)   # pass 'this'
lw   s0, 0(s0)   # s0 = a.vtbl
lw   s0, 4(s0)   # s0 = a.vtbl[1]
jalr s0          # call to_string
```

the compiler knows the method indexes because the vtables for each class use **prefixing:**

subtypes' vtables are prefixed with their supertype's vtable.

`jalr` works like `jal`, but uses a register as the address of the function to call.

Things to ponder on

- what are **interfaces**, then?
  - a "slice" of a vtable, sort of!
  - when a class implements an interface, those methods are present in its vtable at runtime.
- what about **public/protected/private? final? override?**
  - these are all **semantic;** they have no effect on the implementation.
- what about **static** variables/methods?
  - they're globals whose names happen to be scoped inside a class.
- what about **`null`?**
  - we could have **nullable reference types,** like a restricted form of Rust's **`Option<T>`** that only works for references.
- what about inheriting from **multiple classes?**
  - don't even go there. nooooooope nope nope